[Khanh Pham] Linux User-Kernel Communication

Linux User-Kernel Communication

In this document, I will write about special information when we communicate between userspace and kernel

Syscall function

Normally, We often call system call by wrapper function in C library like `write`, `read`, `fork`. Besides, we can also use another function to call system call: `syscall`

With `syscall` function, we call call all system call function just by using only 1 interface. For example, I try using this function to call `fork`:

```
int main()
{
    if (syscall(SYS_fork) == 0)
    {
        printf("child, pid = %d\n", getpid());
        syscall(SYS_exit,0);
    }
        printf("parent, pid = %d\n", getpid());
}
```

The result is same when we call systemcall function in C lib:

```
parent, pid = 9695
child, pid = 9696
```

The reason of using `syscall` function: Only use when we need use some system call which **don't have wrapper interface in C library**.

Now let write about some information about system call to deep understand:

ioctl function

Normally, we offen communicate with kernel module by using read and write in a file. However, if we need to more than transmitting data payload, we need request or command. Then we must parse a request payload in driver like 4 first bytes are request, next 20 bytes is payload,...

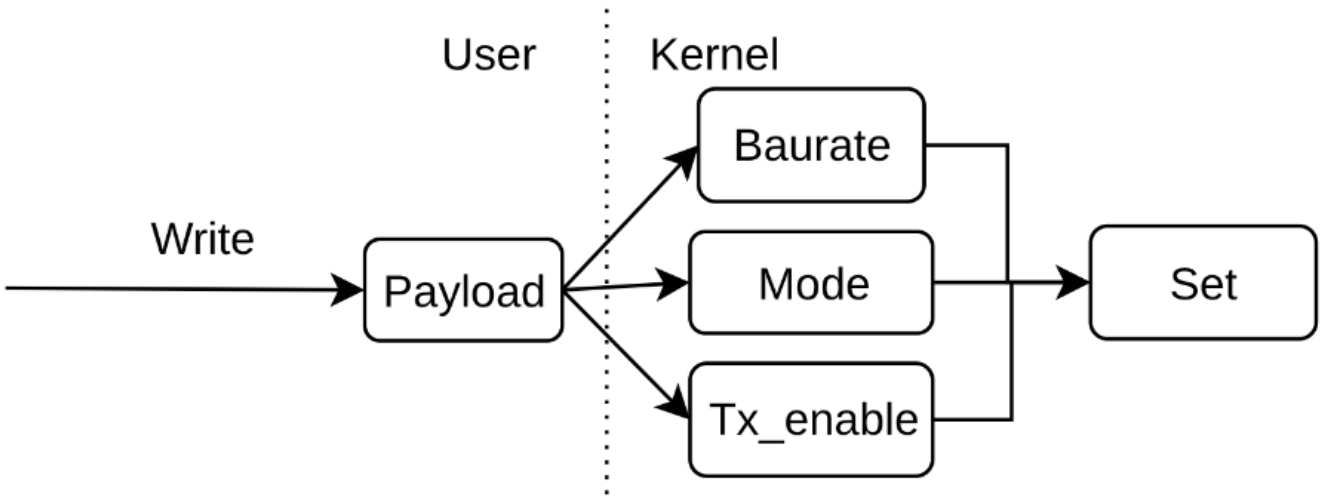
> This way is quite difficut to use and understand.

In this chapter, I will write about another way to solve this problem: `ioctl` function

ioctl is the systemcall function can help us send data to file as a request payload. That mean they have name(ID).

For example: We need a driver to set and get config of uart usb. 3 important arguments is : `baurate` , `mode` and `tx_mode`

- If we use the traditional method: write to set and read to get, we need to parse the data payload when set.



For example:

```
static int g_baud = 9600;
static int g_mode = 0;
static int g_tx = 0;

static ssize_t dev_write(struct file *file, const char __user *user_buf, size_t count, loff_t
*ppos) {
    char kbuf[64];
    int baud = 0, mode = 0, tx = 0;
    size_t copy_size = count < sizeof(kbuf) - 1 ? count : sizeof(kbuf) - 1;

    if (copy_from_user(kbuf, user_buf, copy_size))
        return -EFAULT;

    kbuf[copy_size] = '\0';

    if (sscanf(kbuf, "BAUD=%d,MODE=%d,TX=%d", &baud, &mode, &tx) == 3) {
        g_baud = baud;
        g_mode = mode;
        g_tx = tx;
        pr_info("STR_DRIVER: Succesfull parse-> Baud: %d, Mode: %d, TX: %d\n", g_baud, g_mode,
g_tx);
    } else {
        pr_err("STR_DRIVER: Incomplete parse!\n");
        return -EINVAL;
    }

    return count;
}

static ssize_t dev_read(struct file *file, char __user *user_buf, size_t count, loff_t *ppos)
{
    char kbuf[64];
    int len;

    if (*ppos > 0) return 0;

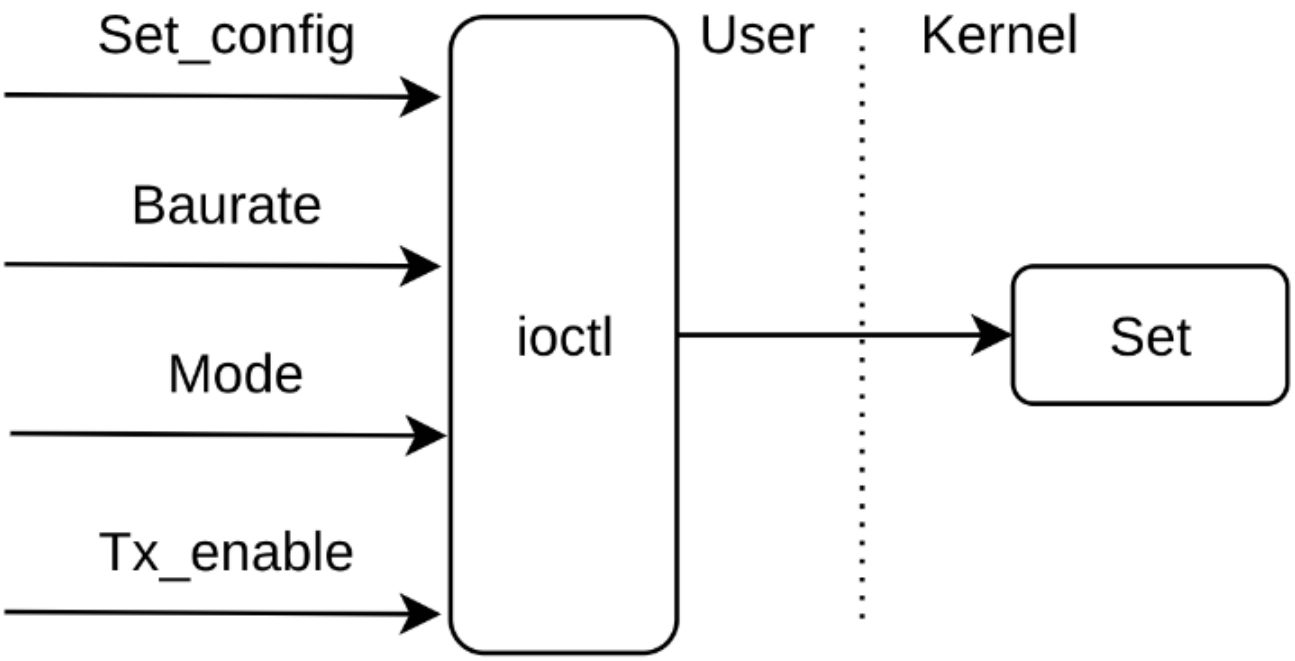
    len = snprintf(kbuf, sizeof(kbuf), "BAUD=%d,MODE=%d,TX=%d\n", g_baud, g_mode, g_tx);
```

```
    if (copy_to_user(user_buf, kbuf, len))
        return -EFAULT;

    *ppos += len;
    pr_info("STR_DRIVER: Sent to userspace\n");
    return len;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .write = dev_write,
    .read = dev_read,
};
```

- If we use ioctl way, we just need define the request command from user space and a struct for data payload. That all!



This is source of the above example with `ioctl` function:

```
static struct dev_config g_cfg = {
    .baud_rate = 9600,
    .mode = 0,
    .enable_tx = 0
```

```
};

static long dev_ioctl(struct file *file, unsigned int cmd, unsigned long arg) {
    switch (cmd) {
        case SET_CONFIG: {
            if (copy_from_user(&g_cfg, (struct dev_config __user *)arg, sizeof(g_cfg))) {
                return -EFAULT;
            }
            pr_info("IOCTL_DRIVER: SET -> Baud: %d, Mode: %d, TX: %d\n",
                    g_cfg.baud_rate, g_cfg.mode, g_cfg.enable_tx);
            break;
        }

        case GET_CONFIG: {
            if (copy_to_user((struct dev_config __user *)arg, &g_cfg, sizeof(g_cfg))) {
                return -EFAULT;
            }
            pr_info("IOCTL_DRIVER: Sent struct to user sapce\n");
            break;
        }

        default:
            return -ENOTTY;
    }
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = dev_ioctl,
};
```

This is the advanate of `ioctl` with traditional method:

ioctl	traditional
Need only copy data from userspace to kernelspace	Need to copy and parse data between user and kernel
Don't need convert between string and interger	Need convert in each layer
Can check the data error in userspace	Always receive data to kernel before checking data
Easy to maintain and extend source code	Harder than using ioctl

proc file

> Linux is a OS that have a definition: **Everything is file**. Therefore, we need to make everything that managing by kernel become file too. That is reason why we have `/proc` filesystem in linux

`/proc` is a dictionary contain some main infomation items which is managed by kernel:

- Information of running process: `/proc/PID`
- Information of process that run command: `/proc/self`
- Thread information in each process: `/proc/PID/task`
- Other system-wide information: `/proc/[a-z]`

With `/proc`, we can easily get/set kernel data. /proc is temporary file, that mean we will see the size of dictionary or file is 0 byte and if we turn off device this folder will be disapeared

For example: I will change the name of process during running time by using the entry: `/proc/self/comm`

```
int main(void)
{
    const char *comm = "/proc/self/comm";

    char result[50];
    int fd = open(comm, O_RDWR);
    int n = read(fd, result, sizeof(result) - 1);
    result[n] = '\0';
    printf("current process name: %s\n", result);
    getchar();

    printf("change process name to khanhne\n");
    char *temp = "khanhne";
    write(fd, temp, sizeof(temp));
    getchar();
}
```

The result of the command `pstree -p`:

```
|-gnome-terminal-(4285)-+-bash(6825)---test(9423)
```

```
|-gnome-terminal-(4285)-+-bash(6825)---khanhne(9423)
```

As we can see, we can change the name of 1 process during running time if we use the proc entry of this process.

> So, after understanding about 3 ways to communicate with kernel from userspace, we can list to summary table bellow:

Type	Detail	Example
Syscal	Use when we need to make kernel do something that userspace can't.	syscal: fork, sleep,getpid,...
ioctl	Use when we need more than writing and reading data payload of a file. Therefore we can reduce some redundant action like parsing string, converting string to char,number	ioctl: speed, power, getdata,...
proc entry	Use when we need to get/set data that kernel manage. That mean we just want to get/set data that exist before, without creating anything new.	name of running process, cpu wasting, ram consumer

netlink

Now we have a problem, what happen when a process want to receive a event from kernel. We can't use syscall because there is no system call function help alert from kernel to userspace.

Moreover, if we use filesystem like `/proc`, we must to polling, that mean process always waste cpu resource to check data.

> We can use netlink in this situation because netlink help process receive event from kernel as a socket fd. As a result, we can use poll/epoll to check without wasting CPU resource. Moreover, netlink also support send multicast event too.

Netlink is a IPC between userspace and kernel space. We use netlink as a normally socket IPC that studied by the IPC document.

```
socket(domain, type, family)
```

With netlink, we must use the `AF_NETLINK` domain argument.

Netlink is a datagram-oriented service so type must be `SOCK_DGRAM` or `SOCK_RAW`

Family is a purpose of this socket: For example:

- `NETLINK_ROUTE`(ip link, ip addr): Receives routing and link updates and may be used to modify the routing tables (both IPv4 and IPv6), IP addresses, link parameters, neighbor setups, queueing disciplines, traffic classes, and packet classifiers
- `NETLINK_NETFILTER`(nftables): Netfilter subsystem.
- `NETLINK_SOCK_DIAG`(ss, netstat): Query information about sockets of various protocol families from the kernel

After creating a new socket for netlink, we can call bind function in some situation:

	Need bind	**Don't need bind**
Type of communication	Multicast(kernel - N process)	Unicast(Kernel - 1 process)
Purpose	Automatically listen events from kernel in group	Request and receive response

An example of using netlink to monitor the new rule of route table

```
#define BUFFER_SIZE 4096

int main() {
    int fd = socket(AF_NETLINK, SOCK_RAW, NETLINK_ROUTE);
    if (fd < 0) {
        perror("socket creation failed");
        return 1;
    }

    // Subscribe to IPv4 route change events
    struct sockaddr_nl sa;
    memset(&sa, 0, sizeof(sa));
    sa.nl_family = AF_NETLINK;
    sa.nl_groups = RTMGRP_IPV4_ROUTE;

    if (bind(fd, (struct sockaddr *)&sa, sizeof(sa)) < 0) {
        perror("bind failed");
        close(fd);
        return 1;
    }
```

```
printf("Listening for new route events...\n\n");

char buf[BUFFER_SIZE];
while (1) {
    // Blocking call: sleep until Kernel pushes a route event (0% CPU)
    int len = recv(fd, buf, sizeof(buf), 0);
    if (len < 0) {
        perror("recv failed");
        break;
    }

    struct nlmsghdr *nh = (struct nlmsghdr *)buf;
    for (; NLMSG_OK(nh, len); nh = NLMSG_NEXT(nh, len)) {
        // Filter for new route creation events
        if (nh->nlmsg_type == RTM_NEWROUTE) {
            struct rtmsg *r = (struct rtmsg *)NLMSG_DATA(nh);

            // Ignore non-IPv4 routes
            if (r->rtm_family != AF_INET) continue;

            char dst_ip[INET_ADDRSTRLEN] = "0.0.0.0";
            char gateway_ip[INET_ADDRSTRLEN] = "N/A";
            char ifname[IF_NAMESIZE] = "N/A";

            struct rtattr *rta = (struct rtattr *)RTM_RTA(r);
            int rtl = RTM_PAYLOAD(nh);

            // Parse route attributes (TLV)
            for (; RTA_OK(rta, rtl); rta = RTA_NEXT(rta, rtl)) {
                if (rta->rta_type == RTA_DST) {
                    inet_ntop(AF_INET, RTA_DATA(rta), dst_ip, sizeof(dst_ip));
                } else if (rta->rta_type == RTA_GATEWAY) {
                    inet_ntop(AF_INET, RTA_DATA(rta), gateway_ip, sizeof(gateway_ip));
                } else if (rta->rta_type == RTA_OIF) {
                    unsigned int ifindex = *(unsigned int *)RTA_DATA(rta);
                    if_indextoname(ifindex, ifname);
                }
            }
```

```
                printf("[NEW ROUTE ADDED] Destination: %s/%d | Gateway: %s | Interface: %s\n",
                       dst_ip, r->rtm_dst_len, gateway_ip, ifname);
            }
        }
    }

    close(fd);
    return 0;
}
```

Result:

```
sudo route add -net 10.0.0.81 gw 10.0.0.1 netmask 255.255.255.255 dev wlp0s20f3

// Monitor process
Listening for new route events...

[NEW ROUTE ADDED] Destination: 10.0.0.81/32 | Gateway: 10.0.0.1 | Interface: wlp0s20f3
```

The pros of netlink with other kernel-user IPCs:

- `Kernel-pushed updates`: Instead of waiting for an application to ask, the kernel can send data when something changes. This means monitoring tools get accurate information as events happen, not on the next polling cycle.
- `Fewer repeated queries`: Because the kernel can notify applications of state changes, there is no need to continuously re-query for data that has not changed, reducing unnecessary load.
- `Structured data responses`: Netlink returns data in defined message formats that applications can parse directly, eliminating the need to extract information from text-based files like those in /proc.
- `Event-driven communication`: Applications can subscribe to specific kernel events and receive notifications only when relevant changes occur, rather than running constant background checks.

> In linux, there are many tools which use netlink like: ip, ss, iproutes

---

Revision #2
Created 2026-10-16 03:07:18 UTC by Khanh Pham
Updated 2026-10-19 09:45:22 UTC by Khanh Pham